Loops in Java

Why Do We Need Loops?

In programming, many tasks need repetition.

Examples:

- Printing numbers
- Processing array elements
- Running the same logic multiple times

Writing the same code again and again is not practical.

Loops allow us to **repeat a block of code** automatically based on a condition.

What is a Loop?

A **loop is a control structure that executes a block of code repeatedly** until a condition becomes false.

Core idea of any loop:

- Start execution
- Check condition
- Execute code
- Update value
- Repeat
- Stop when condition fails

All loops in Java follow this logic.

Why Loops are Important for Beginners

Loops help:

- Reduce code duplication
- Improve readability
- Work easily with arrays and collections

If you understand loops well, many Java topics become easier.

Types of Loops in Java

Java provides **three main looping mechanisms**, but one of them has two forms.

Loops in Java:

- for loop
- enhanced for loop (for-each loop)
- while loop
- do-while loop

Each loop exists for a specific purpose.

Normal for Loop

When to Use Normal for Loop

Use this loop when:

- Number of iterations is known
- Index control is required
- Values need to be modified

This is the most flexible loop in Java.

Syntax of Normal for Loop

```
for (initialization; condition; update) {
    // code to repeat
}
```

Meaning:

- initialization - starting point
- condition - decides loop execution
- update - changes loop value

Simple Normal for Loop Example

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Explanation:

- **int i = 1** - loop starts from 1
- **i <= 5** - loop runs till 5
- **i++** - increases value by 1
- **println(i)** - prints current value

Think of it as:
“Start from 1, go till 5, increase by 1”.

Important Things to Remember (Normal for Loop)

- Initialization runs only once
- Condition is checked before every iteration
- Update runs after each iteration
- Loop stops automatically

Enhanced for Loop (for-each Loop)

What is Enhanced for Loop?

The **enhanced for loop** is a simplified version of the for loop.

It is used mainly for:

- Reading elements from arrays
- Traversing collections

It reduces complexity by removing index handling.

Syntax of Enhanced for Loop

```
for (datatype variable : array) {
    // code
}
```

Meaning:

- datatype - type of element
- variable - holds one value at a time
- array - data source

Java automatically moves to the next element.

Simple Enhanced for Loop Example

```
int[ ] numbers = {10, 20, 30};
for (int n : numbers) {
    System.out.println(n);
}
```

Explanation:

- **numbers** - array being read
- **n** - stores each element one by one
- Loop ends after last element

No index is required here.

When to Use Enhanced for Loop

Use it when:

- You only want to read values
- Index is not required

This loop is very beginner-friendly.

Limitations of Enhanced for Loop

Enhanced for loop **cannot**:

- Modify array elements
- Delete elements
- Access index values

---

while Loop

When to Use while Loop

Use while loop when:

- Number of iterations is not known
- Loop depends on a condition
- Condition changes dynamically

Syntax of while Loop

```
while (condition) {
    // code to repeat
}
```

Condition is checked **before execution**.

Simple while Loop Example

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Explanation:

- **i** is initialized before loop
- Condition is checked first
- Update happens inside loop

If condition is false initially, loop does not run.

Things to Remember (while Loop)

- Initialization must be outside loop
- Update must be inside loop
- Condition must become false

---

do-while Loop

Why do-while Loop Exists

Sometimes code must run **at least once**, regardless of condition.

Java provides do-while for such situations.

Syntax of do-while Loop

```
do {
   // code
} while (condition);
```

Condition is checked **after execution**.

Simple do-while Loop Example

```
int i = 1;
do {
   System.out.println(i);
   i++;
} while (i <= 5);
```

Explanation:

- Code runs first
- Condition is checked later
- Loop runs minimum one time

Semicolon after while is mandatory.

Things to Remember (do-while Loop)

- Executes at least once
- Condition checked at end
- Useful for menu-based logic

Infinite Loops

An **infinite loop** is a loop that never stops.

Causes:

- Condition always true
- Update missing or incorrect

Infinite loops freeze programs.

Always ensure:

- Condition eventually becomes false

---

break Statement in Loops

break is used to:

- Exit loop immediately
- Stop further iterations

Once break executes, loop terminates.

---

continue Statement in Loops

continue is used to:

- Skip current iteration
- Move to next iteration

Loop does not stop.

Loops and Arrays (Concept)

Arrays store multiple values.

Loops help:

- Access each element
- Process data efficiently

Understanding loops makes arrays easier.

Beginner Mistakes to Avoid (Summary)

- Wrong loop condition
- Missing update

- Infinite loops
- Using wrong loop type
- Misusing enhanced for loop

Mistakes are part of learning.

Key Points to Remember

- Loops repeat code
- Condition controls loop
- Java has two for loops
- Normal for gives full control
- Enhanced for is read-only
- while is condition-based
- do-while runs once minimum